

Computer Architecture: *Performance & Power*

Dr. Colin O'Flynn

Lectures

- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How does the machine run? (Control Flow)
- How does the machine store, access, and control data?
- How does the machine execute instructions?
- How is execution made more efficient?
- ***How do we measure execution? What is our limit of performance?***

How to measure performance?

- Clock speed?
- Instructions Per Second?
- Instructions Per Cycle?
- Power Used?
- Something Else?

Clock Speed



**JADE COMPUTER
SUPER-386**

Configuration	Price
20 MHz	\$1498
25 MHz	\$1598
25 MHz CACHE	\$1998
33 MHz Cache	\$2398

Full Featured Professional Systems

- True 20, 25 or 33 MHz 80386 CPU
- 1 MB or 32 BIT RAM Expands to 6 MB
- 384K Shadow RAM
- 1.2 MB 5¼" or 1.44 MB 3½" Floppy Disk Drive
- Fast 1:1 Interleave Dual Hard Disk/ Dual Floppy Disk Controller
- 80387 Coprocessor Socket
- Full Size Case with 5 Half Height Drive Bays
- 101 Key Enhanced Keyboard
- 200 Watt Power Supply
- Built-in Clock/Calendar

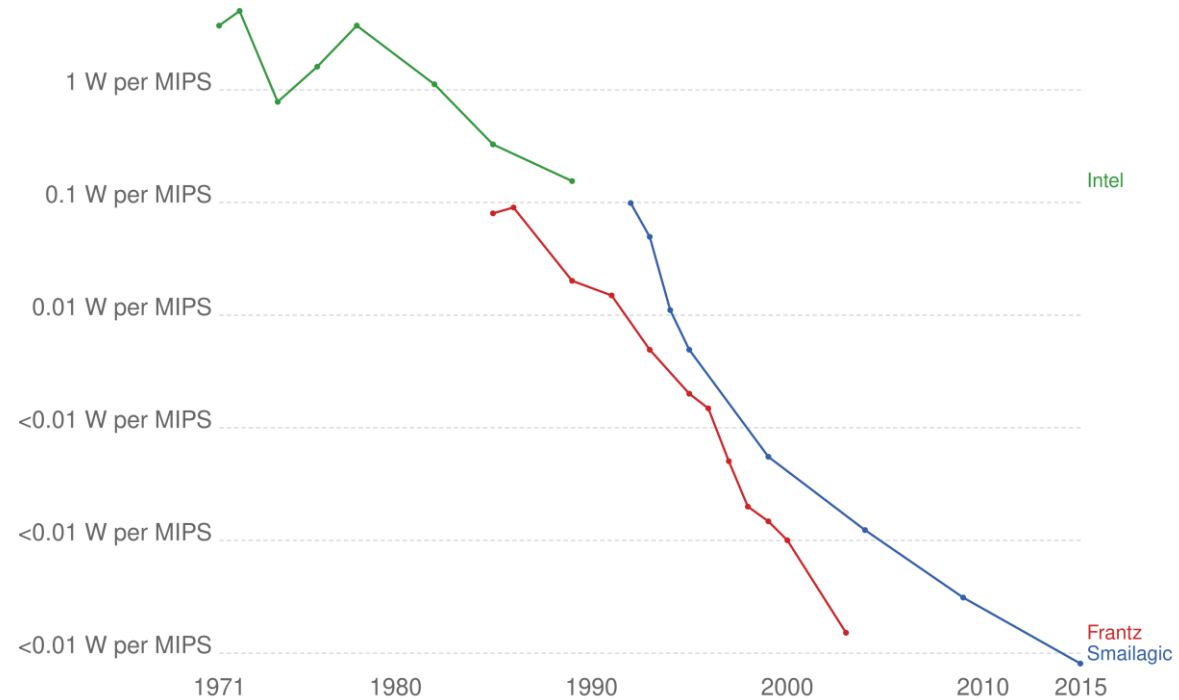


Instructions Per Second

- Rather than pure Hz
– measure how many instructions happen in that time.
- Slightly better, but no idea *what* those instructions do.

Computing efficiency

Computer processing efficiency, measured as the number of watts needed per million instructions per second (Watts per MIPS).



Source: Ray Kurzweil (2005, updated). The Singularity is Near.

CC BY

Considerations around Performance

- We said memory latency can be a driving factor in performance
- Consider different workloads:
 - Sorting a large dataset
 - Calculating a 3D render path
 - Calculating the digits of Pi

What are we trying to benchmark anyway?

- Evaluating performance as a user:
 - We care about benchmark or results (\$ / performance)
- Evaluating performance as a developer / engineer:
 - We care about our program (instrumenting, etc.)
 - This is normally called *profiling*

Workload-Based

- Most useful benchmarks have some somewhat relatable workload

Common Benchmarks

- Dhrystone:
 - Old integer benchmark, not a realistic workload
- Whetstone:
 - Float-point focused, tries to improve on Dhrystone
- EEMBC CoreMark
 - Embedded focused – tries to compare various operations you might do on embedded systems (and most common/useful for us)
- SPEC CPU
 - Desktop/Server CPU evaluation – various more real-life workloads
- PassMark CPU Mark
 - Commercial benchmark tool – widely used in reviews, evaluations, etc

Embedded Performance Measurements

- How do you select a microcontroller?
- Base it on peripherals you need (communication etc), but then what about the code you are running?
 - Machine Learning?
 - Machine Vision?
 - Complex math?
 - How much compute is needed?

CoreMark

- Standard embedded benchmark, various “kernels” you can use:

Segment	Benchmark	Workload / Kernel	URL
802.11 Wi-Fi	IoTMark-Wi-Fi	802.11 connected standby DTIM=10	More info
802.11 Wi-Fi	IoTMark-Wi-Fi	MQTT energy cost	More info
ADAS Performance	ADASMark	Traffic Sign Recognition CNN	More info
ADAS Performance	ADASMark	Debayer, 4 camera @720p HD	More info
ADAS Performance	ADASMark	Dewarp, 4 camera @720p HD	More info
ADAS Performance	ADASMark	Stitch, 4 camera @720p HD	More info
ADAS Performance	ADASMark	Colorspace Conversion, 4 camera @720p HD	More info
ADAS Performance	ADASMark	Gaussian Blur, 4 camera @720p HD	More info
ADAS Performance	ADASMark	Sobel Edge Detection, 4 camera @720p HD	More info
ADAS Performance	ADASMark	Contour Feature Extraction, 4 camera @720p HD	More info
Automotive & Industrial	AutoBench 1.0	Angle-to-Time Conversion	More info
Automotive & Industrial	AutoBench 1.0	Basic Integer and Floating Point	More info
Automotive & Industrial	AutoBench 1.0	Bit Manipulation	More info
Automotive & Industrial	AutoBench 1.0	Cache Buster	More info
Automotive & Industrial	AutoBench 1.0	CAN Remote Data Request	More info

<https://www.eembc.org/products/kernels.php>

CoreMark Results

- E.g., : https://www.eembc.org/viewer/?benchmark_seq=13489

EEMBC Benchmark Score Viewer

Benchmark	
ID	13489
Suite Name	CoreMark
Date Run	2022-06-08
Certified	✓
Certification Date	2022-07-05
Hardware Environment	
Device Vendor	Renesas Electronics*
Device Name and Revision	RX66T
Processor Operating Frequency (MHz)	120.00
Link to Processor Data Sheet (PDF)	Link
Number of Processor Cores	1
Memory configuration	Internal FLASH
Software Environment	
Compiler Name and Version	Renesas CC-RX V3.04
Compiler Flags	-isa=rxv3 -fpu -smap -save_acc -installgn8=loop -optimize=max -goptimize -speed -inline=800
Operating System Name and Version	
Parallel Execution	-
Scores	
CoreMark	720.88
CoreMark/MHz	6.0070

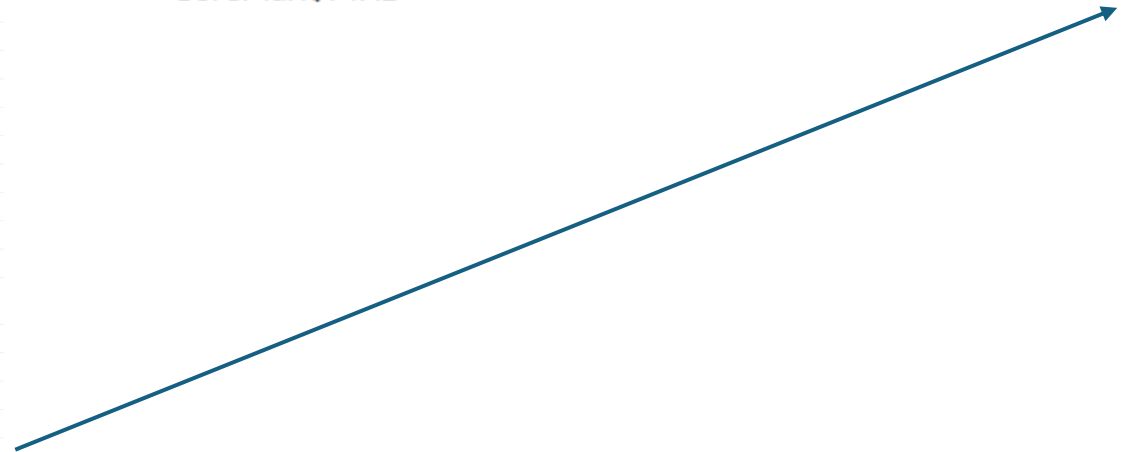
Scores

CoreMark

720.88

CoreMark/MHz

6.0070



Running CoreMark

 Makefile	Makefile: Clean SEPARATE_COMPILE object files too	5 years ago
 README.md	Update README.md - correct typo	11 months ago
 barebones_porting.md	Update barebones_porting.md to avoid vendor specific text.	11 months ago
 core_list_join.c	fixe #42 adding void cast and zero initialization	4 years ago
 core_main.c	docs: Fix a few typos	5 years ago
 core_matrix.c	Ran clang-format with Barr C Embedded Style	6 years ago
 core_state.c	docs: Fix a few typos	5 years ago
 core_util.c	Ran clang-format with Barr C Embedded Style	6 years ago
 coremark.h	Fix typo	3 years ago
 coremark.md5	update files md5	4 years ago

Cor



[+ 4](#)

Lan



<https://github.com/eembc/coremark>

Measuring Cycles on Embedded Targets

- The API at the right uses the Cycle Counter
- This is very handy when comparing execution time on embedded systems – most Arm targets support this!

```
/* Core Debug registers (Cortex-M) */
#define DEMCR      (*(volatile uint32_t*)0xE000EDFC)
#define DWT_CTRL   (*(volatile uint32_t*)0xE0001000)
#define DWT_CYCCNT (*(volatile uint32_t*)0xE0001004)

/* Bit masks */
#define DEMCR_TRCENA (1 << 24)
#define DWT_CYCCNTENA (1 << 0)

volatile uint32_t start, end;

void start_timer(void) {
    /* Enable trace (required for DWT) */
    DEMCR |= DEMCR_TRCENA;

    /* Reset cycle counter */
    DWT_CYCCNT = 0;

    /* Enable cycle counter */
    DWT_CTRL |= DWT_CYCCNTENA;
    start = DWT_CYCCNT;
}

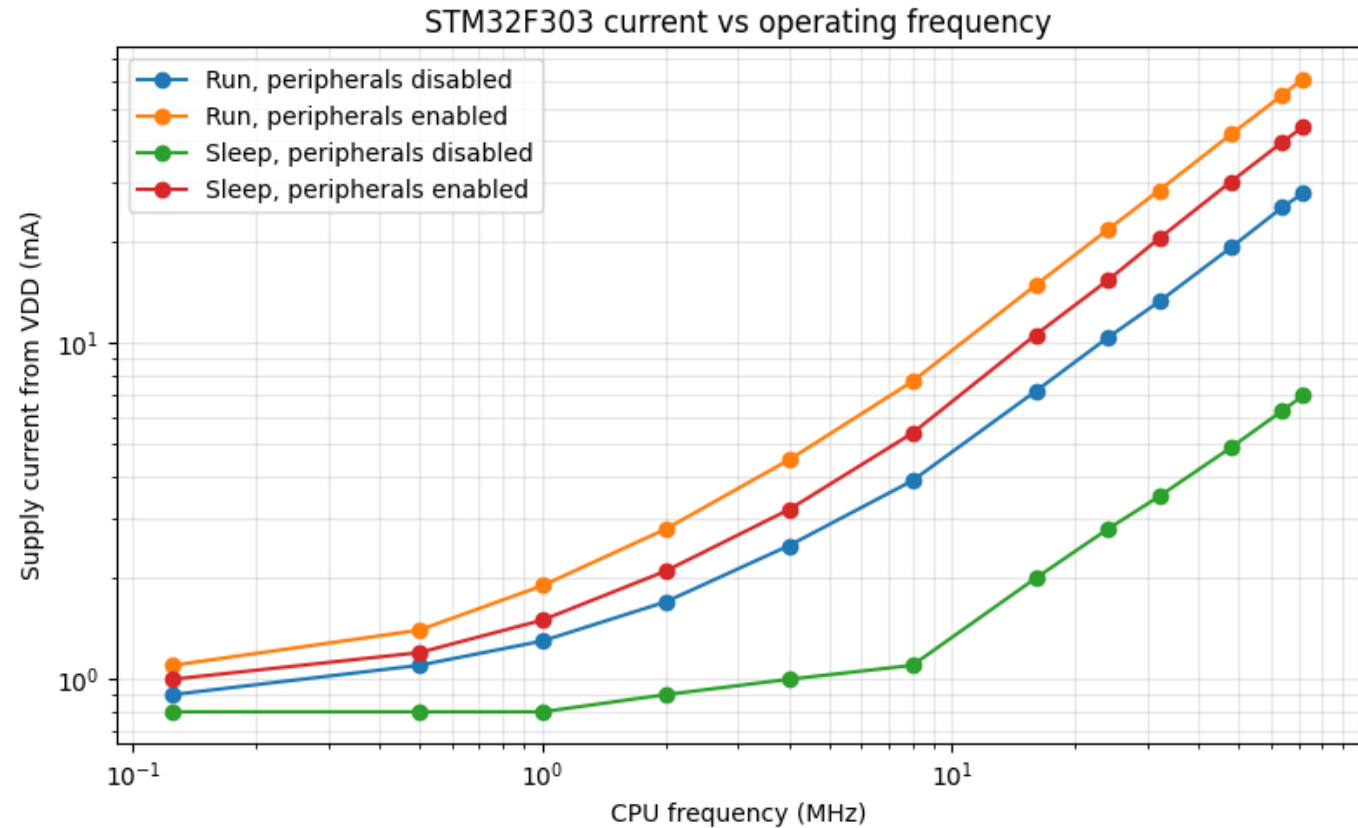
void stop_timer(void) {
    end = DWT_CYCCNT;
}
```

Power Usage

- So far – we've considered mostly selecting a processor based on e.g., price, availability, etc.
- In real life, what is our limiting factor?

→ POWER

Power Usage vs. Performance



Dynamic Power Consumption

- Switching all those transistors that make up our microcontroller mean switching some *capacitance* from a logic-0 to a logic-1
- General formula for power usage with this is:

$$P_{\text{dynamic}} = C \times V^2 \times f$$

C = capacitance

V = Voltage

f = Frequency

Factors Impacting Power Usage

- Capacitance defined by *node size*
 - Newer (smaller) processes have less capacitance as have smaller dies
- Voltage has a major (square) impact
 - Newer (smaller) often run at lower voltages (5V → 3.3V → 1.8V → 1V)
 - Many microcontrollers have a lower core voltage and a higher I/O voltage
- Frequency *should* have a linear impact
 - This is one thing you can easily control

Power Targets

- Power (W) = Instantaneous Consumption
- Energy (J) = Total consumer over time
- Often have some combination of limits of instantaneous power and energy

e.g., CR2032 Coin Cell

PRODUCT DATASHEET

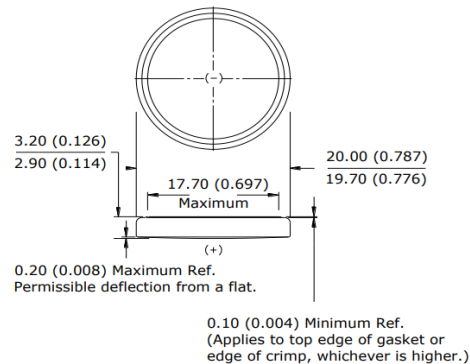
Energizer

ENERGIZER CR2032



Industry Standard Dimensions

mm (inches)



Lithium Coin

Specifications

Classification:	"Lithium Coin"
Chemical System:	Lithium / Manganese Dioxide (Li/MnO ₂)
Designation:	ANSI / NEDA-5004LC, IEC-CR2032
Nominal Voltage:	3.0 Volts
Typical Capacity:	235 mAh (to 2.0 volts) (Rated at 15K ohms at 21°C)
Typical Weight:	3.0 grams (0.10 oz.)
Typical Volume:	1.0 cubic centimeters (0.06 cubic inch)
Max Rev Charge:	1 microampere
Energy Density:	198 milliwatt hr/g, 653 milliwatt hr/cc
Typical Li Content:	0.109 grams (0.0038 oz.)
Operating Temp:	-30C to 60C
Self Discharge:	~1% / year

Safety:



(1) KEEP OUT OF REACH OF CHILDREN. Swallowing may lead to serious injury or death in as little as 2 hours due to chemical burns and potential perforation of the esophagus. **Immediately see doctor; have doctor phone (800) 498-8666.**

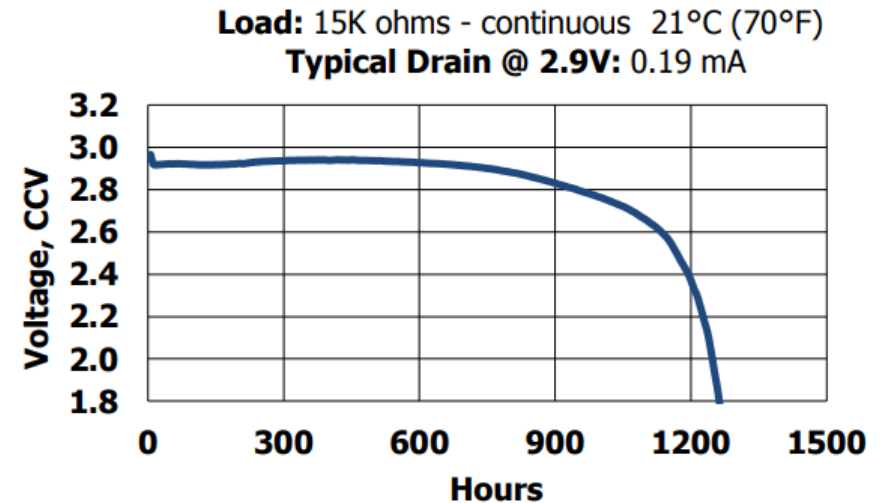
(2) Battery compartment design. To prevent children from removing batteries, battery compartments should be designed with one



<https://data.energizer.com/pdfs/cr2032.pdf>

Simple Battery Power Measurement

Designation:	ANSI / NEDA-3004EC, IEC-CR2
Nominal Voltage:	3.0 Volts
Typical Capacity:	235 mAh (to 2.0 volts) (Rated at 15K ohms at 21°C)
Typical Weight:	3.0 grams (0.10 oz.)



235 mAh meaning:

1 mA for 235 hours

100 uA for 2350 hours

10 mA for 23.5 hours

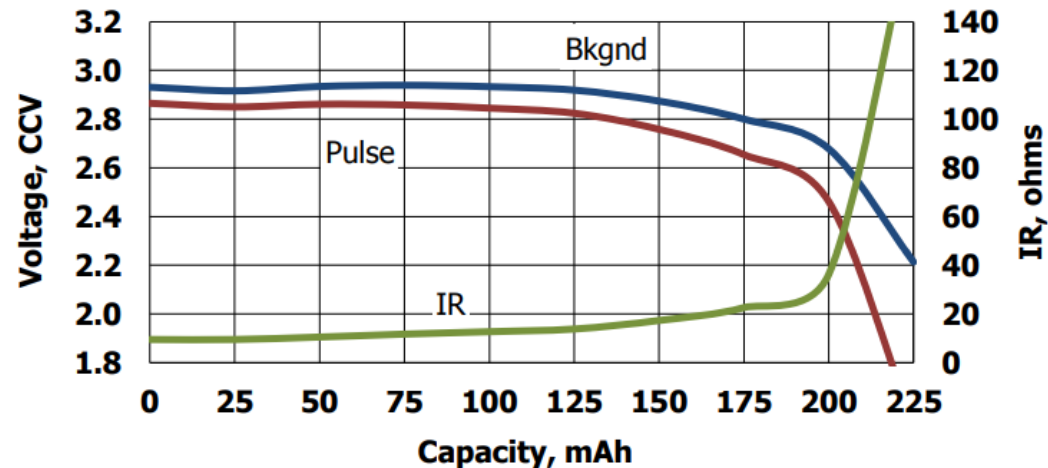
...kinda

Designation: ANSI / NEDA 3004EC, IEC CR2032
Nominal Voltage: 3.0 Volts
Typical Capacity: 235 mAh (to 2.0 volts)
(Rated at 15K ohms at 21°C)
Typical Weight: 3.0 grams (0.10 oz.)

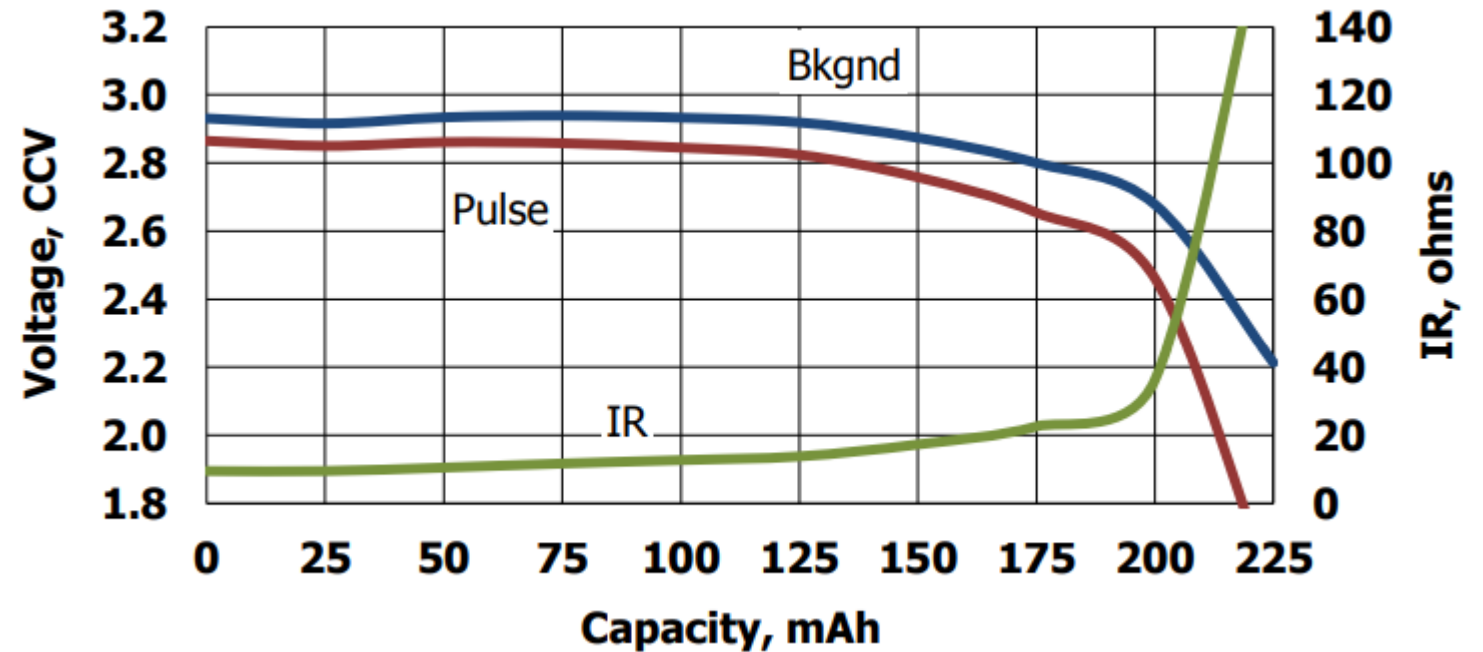
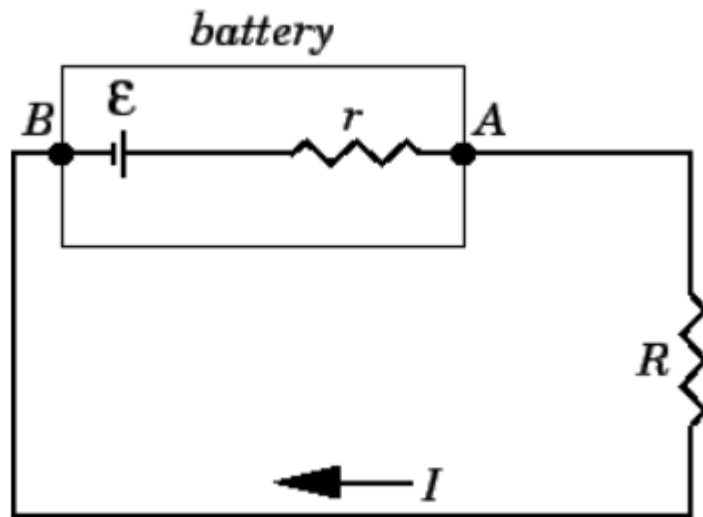
1245

Bkgnd Drain: Continuous
21°C (70°F) 15K ohms
0.19 mA @2.9V

Pulse Drain: 2 seconds X 12 times/day
400 ohms
~6.8 mA @2.7V



Other Consideration: Internal Resistance



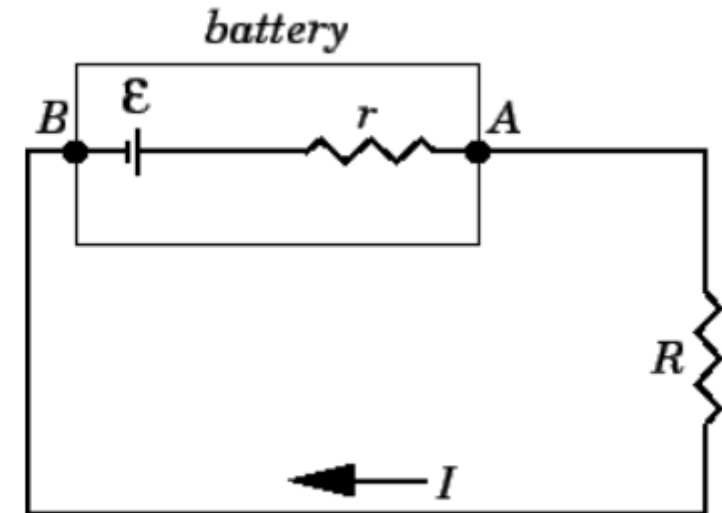
e.g., CR2032 @ 125mAh drawn

- Open circuit (OC) voltage $\sim 2.8\text{V}$
- Internal Resistance (IR) $\sim 18\text{ ohms}$

If drawing 50 mA from battery

$$V = I R = (0.05)(18) = 0.9\text{V}$$

$$V_{\text{load}} = 2.8 - 0.9 = \mathbf{1.9\text{V}}$$



Pulse Usage: Capacitor Banks

- Common to have a capacitor in parallel with battery to have pulse current capability
- This capacitor has a low energy stored however. E.g.:
- Assuming 10uF capacitor @ 2.8V

$$\begin{aligned} E &= \frac{1}{2} \times C \times V^2 \\ &= 0.5 \times 10\text{E-}6 \times 2.8^2 \\ &= 39.2 \text{ uJ} \end{aligned}$$

$$mAh = \text{Joules} / (V \times 3.6)$$

$$\begin{aligned} &= 0.00000778 \text{ mAh} \\ &= 0.028008 \text{ mAs} \end{aligned}$$

Pulse Usage: Explains common sensor cycles

When designing devices powered from small coin cells (think sensors), you often need to use high-energy / sleep cycles.

Not only for total power saving, but to deal with limited total current.

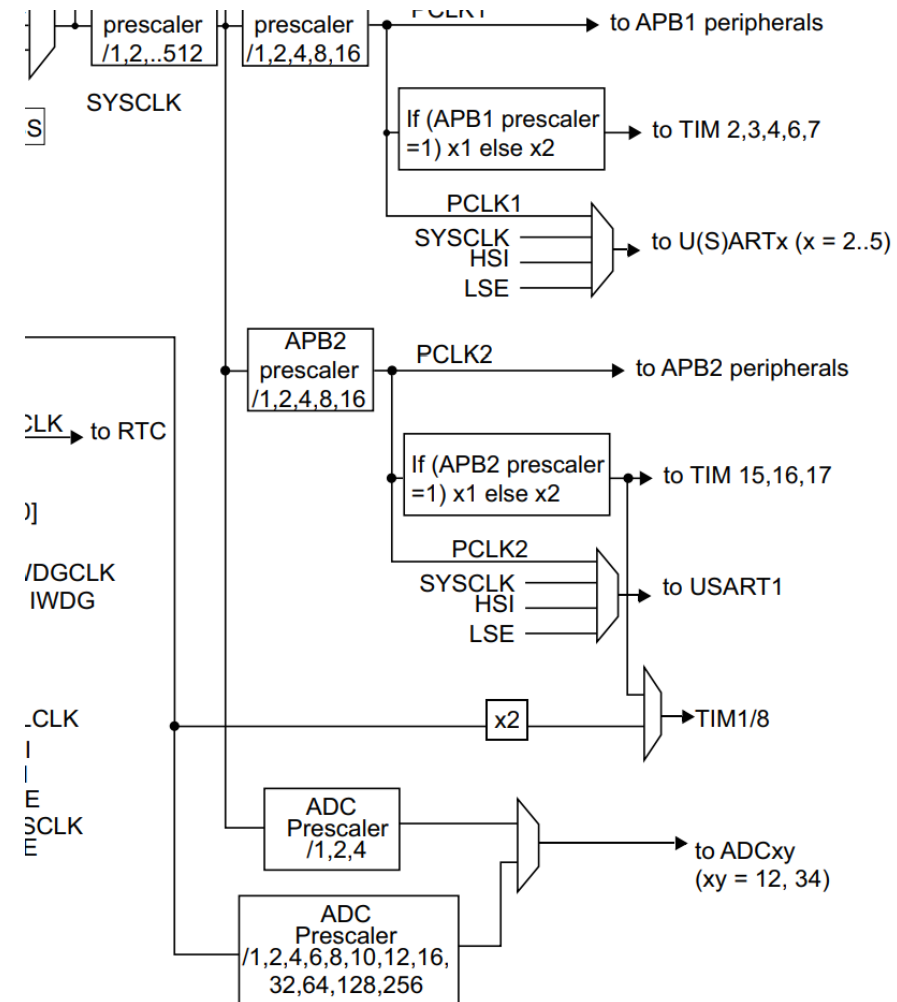


Typical Low-Power Modes

- Most microcontrollers have some combination of:
 - Active (normal) mode
 - Ability to turn on/off peripherals
 - A low-power mode that keeps everything alive (“sleep” normally)
 - SRAM power is ON
 - A super-low power mode that kills most things (“stop” normally)
 - May even turn off SRAM → RAM invalid
 - Often a mode that leaves a “Real Time Clock” (RTC) Active

e.g., STM32F3 Clock Gating

- Most microcontrollers allow you to turn on/off clocks to peripherals
- Many devices have them off by default
- This is a common problem if you have a device not working



e.g., STM32F3 Low-Power Modes

Table 17. Low-power mode summary

Mode name	Entry	wakeup	Effect on 1.8V domain clocks	Effect on V _{DD} domain clocks	Voltage regulator
Sleep (Sleep now or Sleep-on - exit)	WFI	Any interrupt	CPU clock OFF no effect on other clocks or analog clock sources	None	ON
	WFE	Wakeup event			
Stop	PDDS and LPDS bits + SLEEPDEEP bit + WFI or WFE	Any EXTI line (configured in the EXTI registers) Specific communication peripherals on reception events (USART, I2C)	All 1.8V domain clocks OFF	HSI and HSE oscillators OFF	ON or in low-power mode (depends on Power control register (PWR_CR))
Standby	PDDS bit + SLEEPDEEP bit + WFI or WFE	WKUP pin rising edge, RTC alarm, external reset in NRST pin, IWDG reset			OFF

Always check specifics

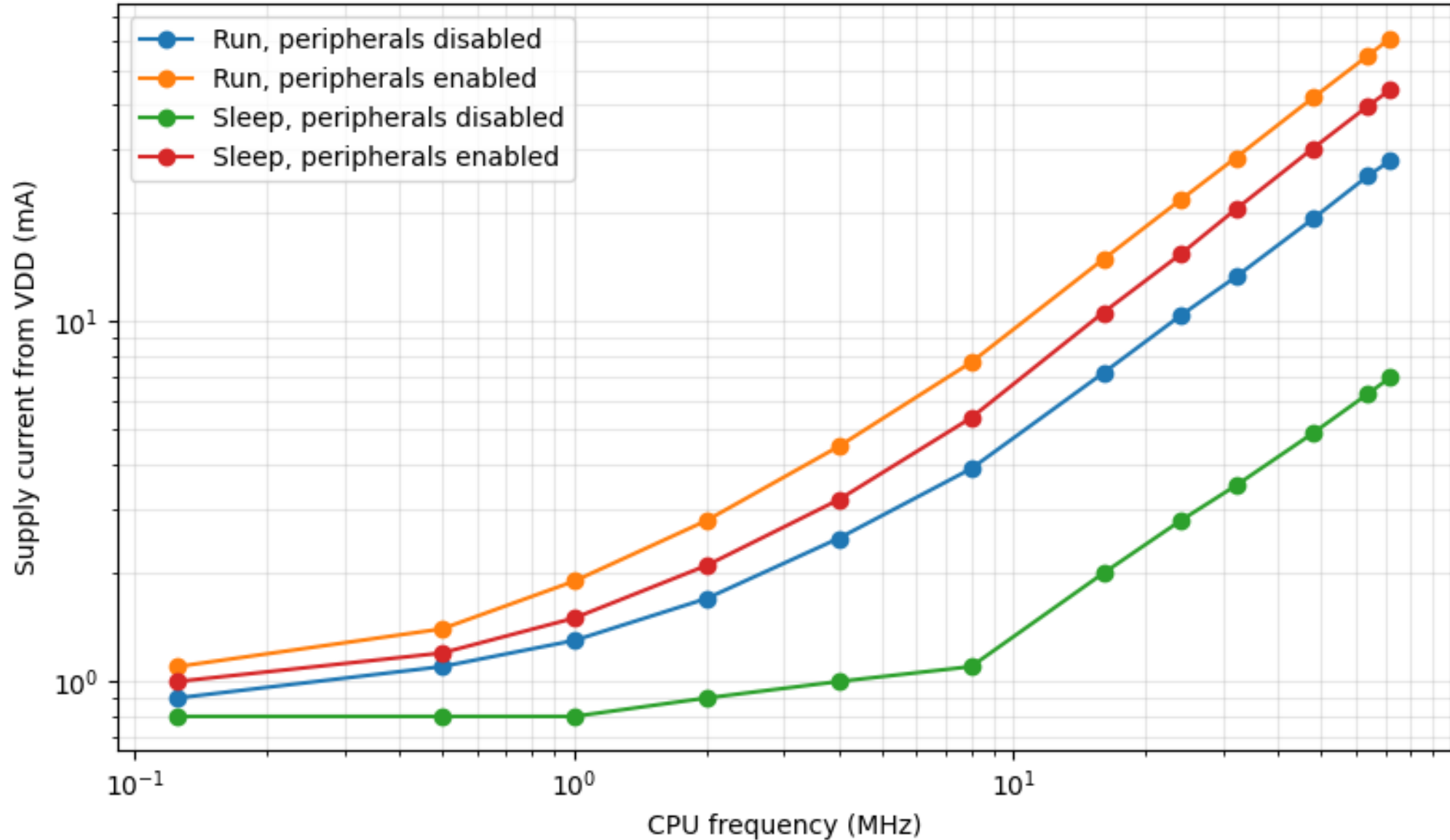
7.3.4 Stop mode

The Stop mode is based on the Cortex-M4 deep-sleep mode combined with peripheral clock gating. The voltage regulator can be configured either in normal or low-power mode in the STM32F3xx devices. In the STM32F3x8 devices, it is meaningless to distinguish between voltage regulator in low-power mode and voltage regulator in Run mode because the regulator is not used and V_{DD} is applied externally to the regulator output. In Stop mode, all clocks in the 1.8 V domain are stopped, the PLL, the HSI, and the HSE RC oscillators are disabled. **SRAM and register contents are preserved.**

7.3.5 Standby mode

The Standby mode allows to achieve the lowest power consumption. It is based on the Cortex-M4 deepsleep mode, with the voltage regulator disabled. The 1.8 V domain is consequently powered off. The PLL, the HSI oscillator and the HSE oscillator are also switched off. **SRAM and register contents are lost** except for registers in the RTC domain and Standby circuitry (see [Figure 8](#)).

STM32F303 current vs operating frequency



STM32F303: Run Mode vs. Freq

Symbol	Parameter	Conditions	f_{HCLK}	Typ		Unit
				Peripherals enabled	Peripherals disabled	
I_{DD}	Supply current in Run mode from V_{DD} supply	Running from HSE crystal clock 8 MHz,	72 MHz	61.3	28.0	mA
			64 MHz	54.8	25.4	
			48 MHz	41.9	19.3	
			32 MHz	28.5	13.3	
			24 MHz	21.8	10.4	
			16 MHz	14.9	7.2	
			8 MHz	7.7	3.9	
			4 MHz	4.5	2.5	
			2 MHz	2.8	1.7	
			1 MHz	1.9	1.3	
			500 kHz	1.4	1.1	
			125 kHz	1.1	0.9	

STM32F303: Sleep Mode vs. Freq

Table 36. Typical current consumption in Sleep mode, code running from Flash or RAM

Symbol	Parameter	Conditions	f_{HCLK}	Typ		Unit
				Peripherals enabled	Peripherals disabled	
I_{DD}	Supply current in Sleep mode from V_{DD} supply	Running from HSE crystal clock 8 MHz,	72 MHz	44.1	7.0	mA
			64 MHz	39.7	6.3	
			48 MHz	30.3	4.9	
			32 MHz	20.5	3.5	
			24 MHz	15.4	2.8	
			16 MHz	10.6	2.0	
			8 MHz	5.4	1.1	
			4 MHz	3.2	1.0	
			2 MHz	2.1	0.9	
			1 MHz	1.5	0.8	
			500 kHz	1.2	0.8	
			125 kHz	1.0	0.8	

Stop & Standby Comparison

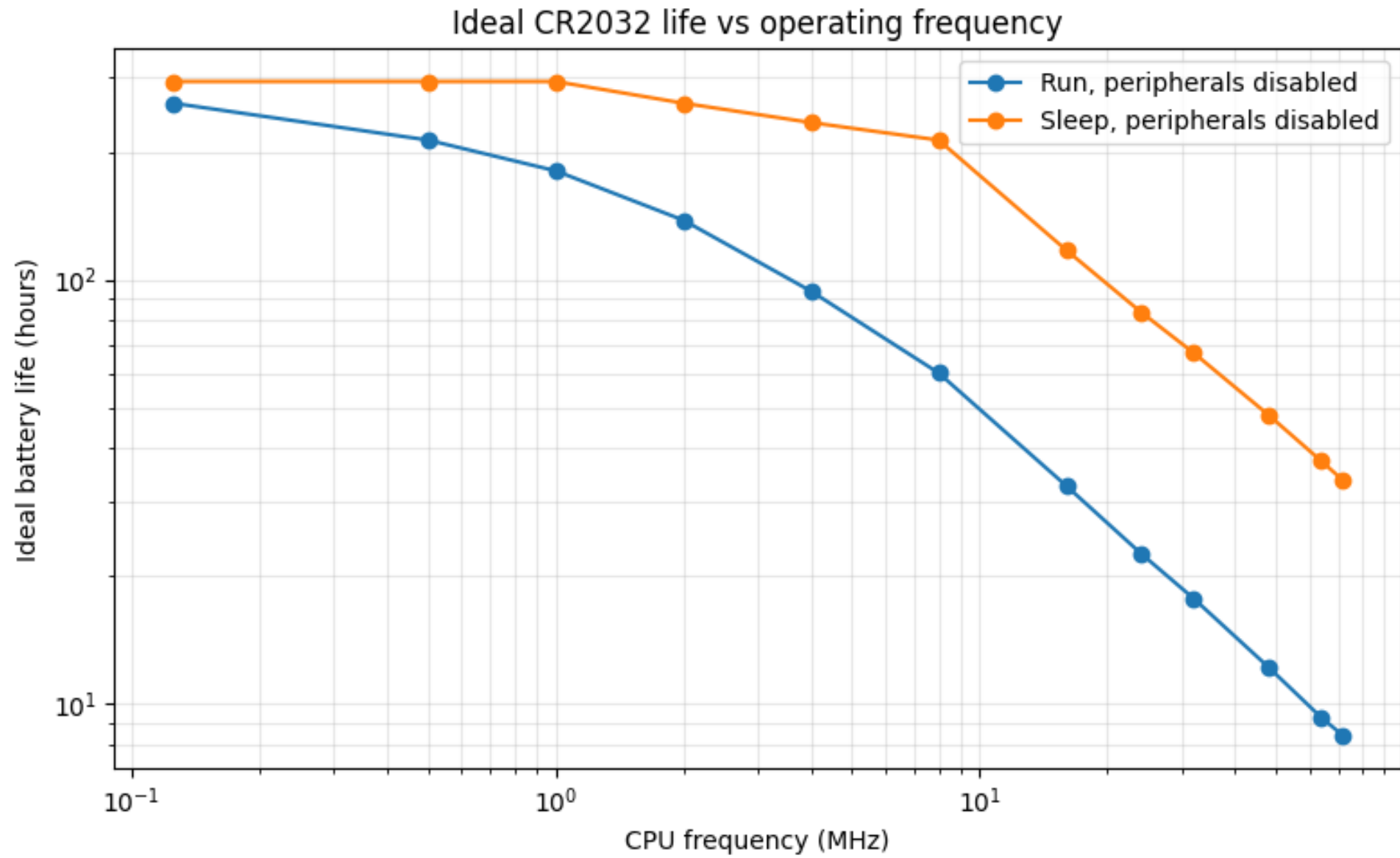
Table 32. Typical and maximum V_{DD} consumption in Stop and Standby modes

Symbol	Parameter	Conditions	Typ @ V_{DD} ($V_{DD}=V_{DDA}$)						Max ⁽¹⁾			Unit
			2.0 V	2.4 V	2.7 V	3.0 V	3.3 V	3.6 V	$T_A = 25\text{ }^{\circ}\text{C}$	$T_A = 85\text{ }^{\circ}\text{C}$	$T_A = 105\text{ }^{\circ}\text{C}$	
I_{DD}	Supply current in Stop mode	Regulator in run mode, all oscillators OFF	20.05	20.33	20.42	20.50	20.67	20.80	44.2 ⁽²⁾	350	735 ⁽²⁾	μA
		Regulator in low-power mode, all oscillators OFF	7.63	7.77	7.90	8.07	8.17	8.33	30.6 ⁽²⁾	335	720 ⁽²⁾	
	Supply current in Standby mode	LSI ON and IWDG ON	0.80	0.96	1.09	1.23	1.37	1.51	-	-	-	
		LSI OFF and IWDG OFF	0.60	0.74	0.83	0.93	1.02	1.11	5.0 ⁽²⁾	7.8	13.3 ⁽²⁾	

CR2032 Battery Callback

- Our 235 mAh battery would last:

Mode	Condition	Current (Typ)	CR2032 Life (<i>Ideal</i>)
Run	72 MHz, Peripherals Disabled	28 mA	8.4 hours
Run	1 MHz, Peripherals Disabled	1.3 mA	181 hours / 7.5 days
Sleep	72 MHz, Peripherals Disabled	7.0 mA	34 hours / 1.4 days
Sleep	0.125 MHz, Peripherals Disabled	0.8 mA	294 hours / 12.2 days
Stop	Regulators in run, oscillators off	21 uA	1.3 years
Standby	IWDG On	1.4 uA	20 years



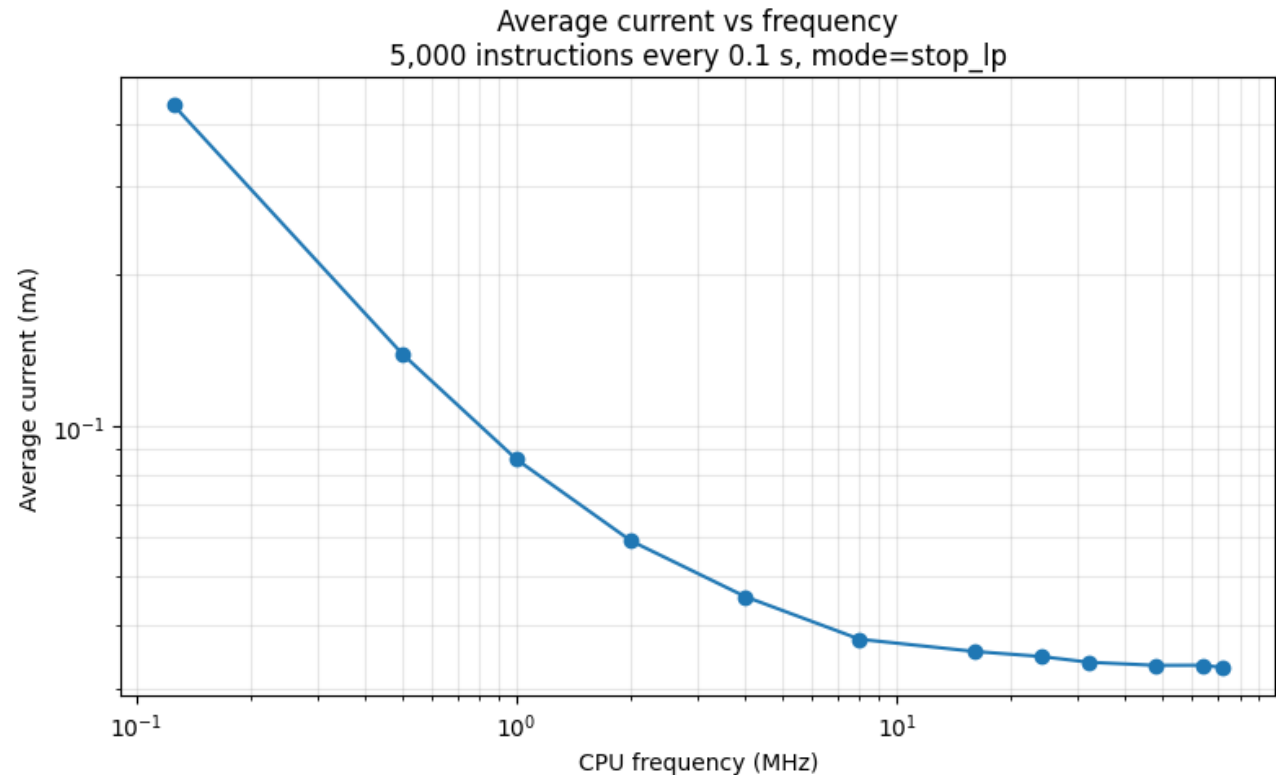
Optimizing for Lowest Total Power

- Most applications require occasional (at least) wakeup to *do something*
- We could “run fast” == higher power, but more done/second
- We could “run slow” == low power, but less done/second

NEVER MAKE ASSUMPTIONS. CHECK FOR DATASHEET DETAILS.

Optimizing for Lowest Total Power

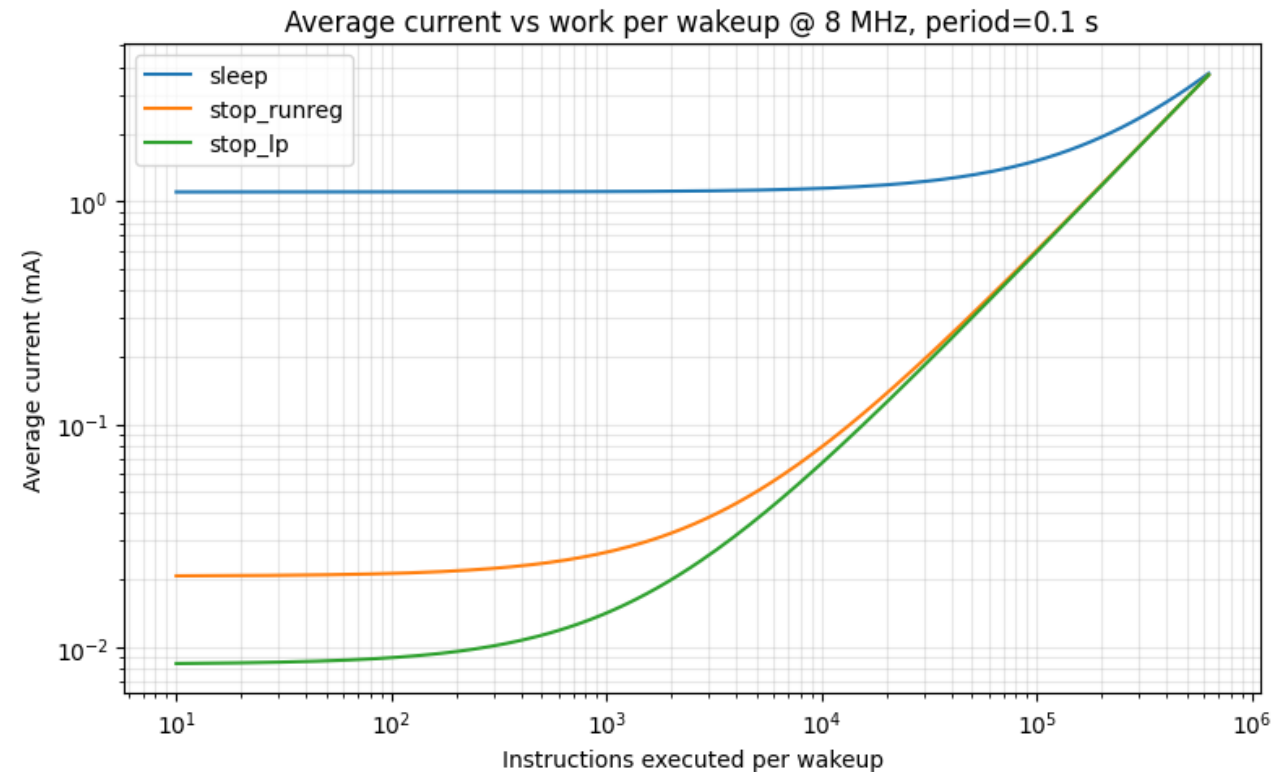
- Also need to consider:
 - Time to enter stop mode
 - Time to wakeup from stop mode
- Graph at right accounts for this & adds power of running for 5000 cycles *at different frequencies*



https://colab.research.google.com/drive/1sOcllVgW9yly8QLc2_68LVZ1Cn8YiFNR?usp=sharing

Optimizing for Lowest Total Power

- Graph at right now shows how effect of wakeup/sleep time should be accounted for!



https://colab.research.google.com/drive/1sOcllVgW9yly8QLc2_68LVZ1Cn8YiFNR?usp=sharing

Targeting Low Power

Very easy to have a small mistake where low-power doesn't work.
For example from STM32F3 datasheet:

Typical and maximum current consumption

The MCU is placed under the following conditions:

- All I/O pins are in input mode with a static value at V_{DD} or V_{SS} (no load)
- All peripherals are disabled except when explicitly mentioned
- The Flash memory access time is adjusted to the f_{HCLK} frequency (0 wait state from 0 to 24 MHz, 1 wait state from 24 to 48 MHz and 2 wait states from 48 to 72 MHz)
- Prefetch in ON (reminder: this bit must be set before clock setting and bus prescaling)
- When the peripherals are enabled $f_{PCLK2} = f_{HCLK}$ and $f_{PCLK1} = f_{HCLK}/2$
- When $f_{HCLK} > 8$ MHz, the PLL is ON and the PLL input is equal to HSI/2 (4 MHz) or HSE (8 MHz) in bypass mode.

Common Low-Power Mistakes

- Ignore effect of pull-up resistors (10K-ohm resistor @ 3.3V = 330uA!)
- Floating inputs
- Not shutting down un-needed peripherals (see per-peripheral contribution from datasheet)
- Remember datasheet values are often typical, *not guaranteed*

Debugging Sleep Problems

- Be warned debugging sleep problems is very annoying
 - Debugger cannot stay attached during debug (clocks are turned off)
 - Need to instrument board carefully to measure low-power
 - Have to think about long-term testing & how to accelerate it (if possible)

Performance & Power Summary

- Analyzing performance is not as simple as you might first think!
- On embedded systems power is often a bigger driver than performance
- We reviewed how to roughly estimate power usage
- We demonstrated how to simulate power usage on STM32F3